ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

OBJECT-ORIENTED PROGRAMMING

PAPER 1 · PRACTICE A

MARKS: 100 **TIME: 3 hours**

BSc Computer Science, 2nd year, semester 1 final examination. Section A: objective questions (10) · Section B: OOP concepts, code tracing, class design and inheritance, interfaces, generics and collections, exceptions and design principles (90).

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (object-oriented programming in Java). It is not a university examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of SIX questions. Answer ALL the questions.
2. Write all code in Java. You do not need to write import statements.
3. Marks are given for correct object-oriented design and logic. Small syntax slips are penalised lightly.
4. Calculators are NOT allowed.
5. Number your answers exactly as the questions are numbered.

SECTION A

QUESTION 1: MULTIPLE CHOICE

Choose the correct answer. Write only the letter (A–D) next to the question number, e.g. 1.11 B.

1.1 Making fields private and giving access through getters and setters is an example of …

A inheritance.
B encapsulation.
C polymorphism.
D overloading.

1.2 Which keyword does a Java class use to inherit from another class?

A implements
B extends
C inherits
D super

1.3 A class that cannot be instantiated and may contain abstract methods is declared …

A final.
B abstract.
C static.
D private.

1.4 Two methods in the same class with the same name but different parameter lists is called …

A overriding.
B overloading.
C hiding.
D casting.

1.5 Which access modifier makes a member visible ONLY inside its own class?

A public
B protected
C private
D (no modifier)

1.6 Which special method runs when an object is created with new?

A The constructor
B The main method
C toString()
D finalize()

1.7 A static field belongs to …

A the class, and is shared by all its objects.
B each object separately.
C the method that uses it.
D the package only.

1.8 Since Java 8, an interface may contain …

A constructors.
B instance fields that each object stores.
C abstract methods and default methods.
D private constructors only.

1.9 Which of these is a CHECKED exception?

A NullPointerException
B ArithmeticException
C IOException
D ArrayIndexOutOfBoundsException

1.10 "A Car has an Engine" describes …

A inheritance.
B composition.
C polymorphism.
D overloading.

[10]

SECTION B

QUESTION 2: OBJECT-ORIENTED CONCEPTS

2.1 Name and briefly explain the FOUR main principles of object-oriented programming. (4)

2.2 Explain the difference between a class and an object. Give an example. (2)

2.3 Explain the difference between method overloading and method overriding. (2)

2.4 Give THREE differences between an abstract class and an interface in Java. (3)

2.5 Explain the difference between an "is-a" and a "has-a" relationship. Give an example of each. (2)

2.6 When comparing two objects, what is the difference between == and equals()? (2)

[15]

QUESTION 3: CODE TRACING

3.1 What is the output of the code below? (3)

```
class Counter {
    private static int total = 0;
    private int count = 0;
    public void click() { count++; total++; }
    public int getCount() { return count; }
    public static int getTotal() { return total; }
}
// in main:
Counter a = new Counter();
Counter b = new Counter();
a.click(); a.click(); b.click();
System.out.println(a.getCount() + " " + b.getCount() + " " + Counter.getTotal());
```

3.2 Consider the code below.

```
class Animal {
    public String sound() { return "..."; }
    public String describe() { return "I say " + sound(); }
}
class Dog extends Animal {
    public String sound() { return "Woof"; }
}
class Puppy extends Dog {
    public String sound() { return super.sound() + "!"; }
}
// in main:
Animal[] zoo = { new Animal(), new Dog(), new Puppy() };
for (Animal x : zoo)
    System.out.println(x.describe());
```

3.2.1 What is the output? (3)

3.2.2 Explain which OOP feature makes describe() print a different sound for each object. (2)

3.3 Consider the code below.

```
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { super(); System.out.print("C "); } }
// in main:
new C();
```

3.3.1 What is the output? (2)

3.3.2 Explain why. (1)

3.4 Consider the code below.

```
static void change(int x, int[] arr, StringBuilder sb) {
    x = 10;
    arr[0] = 10;
    sb.append("!");
}
// in main:
int x = 1;
int[] arr = {1, 2};
StringBuilder sb = new StringBuilder("Hi");
change(x, arr, sb);
System.out.println(x + " " + arr[0] + " " + sb);
```

3.4.1 What is the output? (3)

3.4.2 Explain why x did not change but arr[0] did. (2)

3.5 Consider the code below.

```
try {
    int[] a = new int[3];
    a[3] = 5;
    System.out.println("A");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("B");
} finally {
    System.out.println("C");
}
System.out.println("D");
```

3.5.1 What is the output? (3)

3.5.2 Why is A not printed? (1)

[20]

QUESTION 4: CLASS DESIGN AND INHERITANCE

4.1 Write a class BankAccount with: (12)

- private fields accountNumber (String), owner (String) and balance (double);
- a constructor that takes the account number and owner, and sets the balance to 0;
- getters for all three fields (no setters);
- a method deposit(double amount) that throws an IllegalArgumentException if amount is 0 or less, and otherwise adds it to the balance;
- a method withdraw(double amount) that returns false (and changes nothing) if there is not enough money, and otherwise subtracts the amount and returns true;
- a toString() method that returns, for example: 1001 (T. Nkosi): R250.00

4.2 Write a class SavingsAccount that inherits from BankAccount and has: a private field interestRate (double, e.g. 0.05 for 5%); a constructor that takes the account number, owner and interest rate; a method addInterest() that adds balance × interestRate to the account using deposit(); and a toString() that adds the rate, e.g. 1002 (L. Molefe): R1050.00 at 5.0% (8)

4.3 Write code for main that creates an ArrayList<BankAccount> holding one BankAccount and one SavingsAccount (rate 5%), deposits R1 000 into each account, adds interest ONLY to savings accounts (use instanceof), and prints every account. (6)

4.4 Draw a UML class diagram of BankAccount and SavingsAccount, showing fields, methods, visibility (+ / −) and the inheritance arrow. (4)

[30]

QUESTION 5: INTERFACES, GENERICS AND COLLECTIONS

5.1 The interface Shape declares double area() and double perimeter(). Write the interface, and a class Circle that implements it (Circle has a private field radius set by its constructor). (5)

5.2 The class Student has the fields String name and int mark, and implements Comparable<Student>. Write its compareTo method so that students are sorted from the HIGHEST mark to the lowest. (4)

5.3 Give ONE advantage of using generics, for example ArrayList<String> instead of a raw ArrayList. (2)

5.4 Name the most suitable Java collection for each task and give a reason: (a) storing student numbers so that no number is stored twice; (b) finding a student's name quickly when you know the student number. (4)

[15]

QUESTION 6: EXCEPTIONS AND DESIGN PRINCIPLES

6.1 Explain the difference between checked and unchecked exceptions. Give an example of each. (3)

6.2 Write a checked exception class InsufficientFundsException with a constructor that takes a message. (3)

6.3 Explain the Single Responsibility Principle, and what is meant by low coupling and high cohesion. (4)

[10]

TOTAL: 100

END OF QUESTION PAPER